

2026

F I N A L P R O J E C T P R O P O S A L

Medical AI Prediction System (MAPS)

A comprehensive, role-based medical platform integrating ML.NET disease prediction, ONNX-powered image classification, NLP-driven clinical note processing, personalized AI chatbot with multi-modal inputs, Docker containerization, and Kubernetes orchestration within a unified healthcare ecosystem governed by hierarchical access control.

SUBMITTED BY	Muhammad Ameer Hamza · Muhammad Jazib Asad Kayani · Muhammad Abdullah Khan
SUBMITTED TO	Sir Uzair Janjua · Advanced Programming
CLASS	BSCS-6-B · KICSIT Computer Science Department
DATE	May 14, 2026

AI CHATBOT	DOCKER	KUBERNETES	MULTI-MODAL	ML.NET	OPEN SOURCE
------------	--------	------------	-------------	--------	-------------

Table of Contents

1. Introduction	1
1.1 Project Objectives	1
2. Role-Based Hierarchy System	3
2.1 Role Feature Comparison	3
2.2 Chat Permission Matrix	4
3. System Architecture	4
3.1 Architecture Layers	4
3.2 Technology Stack	5
4. Module A: Admin Panel	7
5. Module B: Doctor Module (Enhanced)	9
6. Module C: Patient Module	14
7. AI/ML Pipeline	15
8. AI Chatbot Architecture	17
9. Database Design	18
10. Docker Containerization	20
11. Kubernetes Orchestration	22
12. Security Architecture	25
13. API Design	26
14. Scope and Expected Outcomes	27
15. Conclusion	29

1. Introduction

The Medical AI Prediction System (MAPS) is a comprehensive, role-based medical platform designed to integrate artificial intelligence into healthcare delivery while maintaining strict professional governance and clinical oversight. This enhanced proposal presents the design, architecture, and implementation plan for MAPS, a system that combines ML.NET-based disease prediction, ONNX-powered medical image classification, NLP-driven clinical note processing, real-time SignalR communication, and a personalized AI chatbot for doctors with multi-modal input support within a unified platform governed by a hierarchical access control model.

MAPS provides two complementary frontends: a WinForms desktop application for clinic-based staff and an ASP.NET Core MVC web application for remote access, both powered by a single shared ASP.NET Core Web API backend. The entire system is containerized using Docker with Docker Compose orchestration for single-command local deployment, and Kubernetes (K8s) for production-grade horizontal scaling, rolling updates, and automated failover — ensuring consistent deployment across development, testing, and production environments. Every component of the technology stack is built on free, open-source tools, eliminating licensing costs and vendor lock-in while maintaining enterprise-grade capabilities through PostgreSQL, Redis, MinIO, Ollama, Grafana Loki, and a full Kubernetes control plane.

MAPS addresses a critical gap in healthcare technology: the absence of unified platforms that combine AI-driven diagnostics with role-appropriate access controls. The Patient Module does not include direct AI chatbot access; all AI-generated insights are routed through assigned doctors to maintain clinical accountability and prevent misinterpretation of diagnostic results.

1.1 Project Objectives

Objective	Description	Success Criteria
Multi-Disease Prediction	Predict Diabetes, Heart Disease, Pneumonia, Brain Tumour, and Skin Cancer through structured, image, and text modalities	Prediction accuracy >85% on validation datasets with confidence scores
AI Chatbot for Doctors	Personalized AI assistant supporting text, image, voice, and document inputs with RAG-enhanced clinical context	Doctors can query in 4 input modalities and receive context-aware clinical suggestions
Differential Diagnosis	Generate ranked lists of possible diagnoses with reasoning chains and confirmatory test recommendations	Top-3 differential matches clinical consensus in ≥80% of test cases
Risk Stratification	Continuous patient risk scoring with four-tier urgency classification and proactive alerting	High-risk patients identified with 90%+ recall; alerts within 60 seconds
Smart Scheduling	Prioritize appointments based on AI risk scores for optimal resource allocation	High-risk patients receive same-day appointments
Clinical NLP	Automatic summarization and entity extraction from free-text clinical notes	Entity extraction F1 score >0.80 on clinical documentation
Docker Deployment	Fully containerized deployment with Docker Compose for one-command infrastructure setup	Complete system deployable via 'docker compose up' with all services healthy within 5 minutes
Kubernetes Orchestration	Production-grade K8s cluster with auto-scaling, rolling updates, and self-healing for all MAPS microservices	Full cluster operational within 10 minutes via Helm chart;

Objective	Description	Success Criteria
		zero-downtime rolling updates verified
Real-Time Communication	SignalR-powered chat between doctors and patients with file sharing and read receipts	Message delivery latency <500ms with persistent history

Table 1: MAPS Project Objectives and Success Criteria

2. Role-Based Hierarchy System

The MAPS platform implements a strict three-tier role hierarchy that governs every aspect of system access, data visibility, and feature availability. Administrators possess unrestricted system oversight including Kubernetes cluster management, Doctors serve as clinical decision-makers with advanced AI tools, and Patients interact through a controlled portal providing health information without direct AI diagnostic access.

2.1 Role Feature Comparison

Feature / Capability	Admin (Tier 1)	Doctor (Tier 2)	Patient (Tier 3)
User Management (CRUD)	Full Access	View Own Profile	View Own Profile
Doctor-Patient Assignment	Full Control	View Assignments	View Assignment
System Analytics Dashboard	Full Access	Department Stats	No Access
AI Model Administration	Full Control	Use Models	No Access
AI Disease Prediction	View All	Create + View Own	Doctor-Shared Only
Differential Diagnosis	View All	Create + View Own	Doctor-Shared Only
Risk Assessment	View All Scores	View Assigned Patients	View Own Score
Clinical Notes	View All	Create + View Assigned	No Access
Medical Image Analysis	View All	Upload + Analyze	No Access
AI Chatbot (Doctor Only)	No Access	Full Access (4 Modalities)	No Access
Smart Appointment Scheduling	Override Priority	View Priority Queue	Book + Reschedule
Doctor-Patient Chat	Chat with Anyone	Chat with Assigned	Chat with Assigned
Prescription Management	View All	Create and Manage	View Own
Health Records	View All Records	View Assigned Records	View Own Records
Docker Config Management	Full Access	No Access	No Access
Kubernetes Cluster Management	Full Access	No Access	No Access

Table 2: Role-Based Feature Comparison Matrix

2.2 Chat Permission Matrix

Initiator	Can Chat With Admin	Can Chat With Doctor	Can Chat With Patient
Admin	Yes (All Admins)	Yes (Any Doctor)	Yes (Any Patient)
Doctor	Yes (Any Admin)	Yes (Assigned Colleagues)	Yes (Assigned Patients Only)
Patient	Yes (Support Channel)	Yes (Assigned Doctor Only)	No

Table 3: Chat Permission Matrix by Role

3. System Architecture

The MAPS platform follows a layered architecture pattern that separates concerns across presentation, business logic, AI/ML processing, chatbot orchestration, and data persistence layers. The entire system is containerized using Docker with Docker Compose orchestration for local/development environments, and Kubernetes (K8s) for production deployment with horizontal pod autoscaling, rolling updates, and self-healing capabilities.

3.1 Architecture Layers

Layer	Components	Responsibility
Presentation Layer	ASP.NET Core MVC / Razor Pages, WinForms Desktop, Blazor WebAssembly, Bootstrap 5, Chart.js, SignalR Client SDK, Chatbot Widget	Server-side rendered UI; native Windows desktop for clinic staff; interactive SPA client; AI chatbot interface for doctors
Application Layer	ASP.NET Core Web API, SignalR Hubs, Controllers, Services, Chatbot Orchestrator, Hangfire Jobs	RESTful API endpoints, business logic orchestration, real-time WebSocket communication, multi-modal chatbot routing, background job scheduling
AI / ML & Chatbot Layer	ML.NET, ONNX Runtime, Ollama LLaMA 3, Whisper.cpp, NLP Custom Pipeline, pgvector RAG	Disease prediction, image classification, multi-modal chatbot with text/image/voice/document inputs, clinical note summarization, voice transcription, medical literature search
Data Layer	PostgreSQL + EF Core + pgvector, Redis Cache, MinIO Object Store, Chatbot Memory	Relational data persistence with ORM, vector similarity search for RAG, session management, medical image/document storage, conversation history
Security Layer	ASP.NET Core Identity + JWT, TLS 1.3, AES-256, Serilog + Grafana Loki	Secure authentication, role-based claims, end-to-end encryption, data at rest encryption, structured logging with alerting
Docker Layer	Docker Compose, Nginx, Multiple containers, Persistent volumes, Health checks	Container orchestration for dev/test, SSL termination, network isolation, data persistence, service monitoring
Kubernetes Layer	Kubernetes Cluster, Helm Charts, HPA, Ingress Controller, ConfigMaps, Secrets, PersistentVolumeClaims	Production orchestration, horizontal pod autoscaling, rolling updates, self-healing, service discovery, secrets management, load balancing

Table 4: MAPS System Architecture — Layered Overview (with Kubernetes)

3.2 Technology Stack

Every technology listed is free and open-source, ensuring zero licensing costs and full community support.

Technology	Purpose	Rationale
C# 12 / .NET 8	Primary language and runtime for all backend and frontend components	Strongly typed, modern OOP; native integration with ML.NET and ONNX Runtime; completely free and open-source

Technology	Purpose	Rationale
ASP.NET Core Web API	Shared backend RESTful API serving both desktop and web clients	Industry-standard REST API framework; unified backend ensures consistent behaviour across all clients
ASP.NET Core MVC	Web frontend with server-side rendered views	Razor views in the same C# ecosystem; free and open-source
WinForms (.NET 8)	Native Windows desktop application for clinic-based staff	Lightweight native desktop UI included in .NET SDK at no cost
Blazor WebAssembly	Interactive SPA client for real-time features	Runs C# in browser via WebAssembly; shares code with backend
Bootstrap 5 + Chart.js	Responsive UI and interactive data visualization dashboards	Most popular free CSS framework; MIT-licensed Chart.js
ML.NET	Disease prediction via structured dataset classification models	Microsoft's free, open-source ML library for C#; no Python runtime needed
ONNX Runtime (C#)	Medical image classification using pre-trained CNN models	Free, open-source inference engine; runs deep learning models in C#
Ollama (LLaMA 3)	AI chatbot engine for multi-modal clinical reasoning	Free, self-hosted LLM; runs LLaMA 3 locally with no API costs; complete data privacy
Whisper.cpp	Voice-to-text transcription for doctor voice notes	Free, open-source speech recognition; fast CPU/GPU inference
SignalR	Real-time bidirectional chat and alert notifications	Built into ASP.NET Core at no cost; supports WebSocket and long polling
PostgreSQL 16 + EF Core	Primary relational database with ORM	Free, open-source enterprise RDBMS; pgvector extension for embeddings
pgvector Extension	Vector similarity search for RAG-enhanced chatbot context	Free PostgreSQL extension; stores and queries LLM embedding vectors
Redis / In-Memory Cache	Session management and performance optimization	Free, open-source in-memory data store; sub-millisecond caching
MinIO Object Storage	Medical image and document file storage	Free, open-source S3-compatible object storage; self-hosted
ASP.NET Core Identity + JWT	Secure authentication with role-based claims	Free, built-in ASP.NET Core authentication; short-lived access tokens
QuestPDF	Professional PDF report generation	Free, open-source MIT-licensed .NET library for programmatic PDF creation
Serilog + Grafana Loki	Structured logging with centralized aggregation	Free, open-source logging pipeline; Grafana for dashboards and alerts
Docker + Docker Compose	Containerization and orchestration for dev/test/staging	Free, open-source container platform; one-command deployment; network isolation
Nginx	Reverse proxy, SSL termination, and load balancing	Free, open-source web server; handles HTTPS termination and routing
Hangfire	Background job scheduling	Free, open-source; persistent job queue in PostgreSQL
FluentValidation	Input validation rules for API request models	Free, open-source validation library; type-safe rules
MediatR	Mediator pattern for decoupled request/response handling	Free, open-source; reduces coupling between controllers and business logic

Technology	Purpose	Rationale
Kubernetes (K8s)	Production container orchestration platform	Free, open-source; CNCF-graduated; industry standard for scalable, resilient microservice deployment
Helm	Kubernetes package manager and deployment templating	Free, open-source; simplifies K8s manifest management; supports environment-specific value overrides
kubectl	Kubernetes CLI for cluster management	Free, official K8s CLI; manages deployments, pods, services, and secrets
Prometheus	Kubernetes-native metrics collection and alerting	Free, open-source; integrates with K8s via ServiceMonitor; powers Grafana dashboards
Horizontal Pod Autoscaler (HPA)	Auto-scaling pods based on CPU/memory metrics	Built into Kubernetes; scales API and AI containers automatically under load
Kubernetes Ingress (Nginx)	HTTP/HTTPS routing and load balancing in K8s cluster	Free, open-source Nginx Ingress Controller; manages TLS termination in production
Kubernetes Secrets + ConfigMaps	Secure configuration and environment management	Native K8s secret management; encrypted at rest in etcd; role-based access
Git + GitHub	Version control and collaborative development	Free distributed version control; GitHub provides free private repositories

Table 5: MAPS Technology Stack — All Free/Open-Source (Kubernetes highlighted in blue)

4. Module A: Admin Panel

The Admin Panel is the command center of MAPS, providing administrators with comprehensive oversight including user lifecycle management, doctor-patient assignment, system analytics, AI model administration, feedback analytics, audit trail, system configuration, and Kubernetes/Docker deployment management. The Admin Panel spans eight sub-sections, with the System Configuration and Security module extended to include Kubernetes cluster health monitoring, pod status dashboards, and Helm release management through an integrated Grafana interface.

5. Module B: Doctor Module (Enhanced)

The Doctor Module is the most feature-rich component of MAPS, comprising sixteen distinct features spanning AI-powered diagnostics, a personalized multi-modal AI chatbot, real-time communication, clinical documentation, voice dictation, drug interaction checking, medical literature search, and intelligent scheduling. The flagship feature is the AI Personalized Chatbot — a doctor-only feature accepting text, image, voice, and document inputs powered by locally-hosted Ollama LLaMA 3 with RAG-enhanced context retrieval.

Disease	Dataset Input	Image Input	Text Prompt
Diabetes	Glucose, BMI, Age	-	Yes
Pneumonia	-	Chest X-Ray	Yes
Brain Tumour	-	MRI Scan	Yes
Skin Cancer	-	Skin Lesion Photo	Yes
Heart Disease	ECG, Cholesterol	-	Yes

Table 6: Supported Diseases and Input Modalities

6. Module C: Patient Module

The Patient Module provides patients with a secure interface for managing healthcare interactions. Features include the Patient Dashboard, Appointment Booking integrated with the Smart Priority Engine, Health Records and History in a timeline view, View AI Predictions (Doctor-Shared Only), and the Feedback System. The Patient Module does not include an AI chatbot — all AI-generated insights are routed through the assigned doctor.

7. AI/ML Pipeline

The MAPS AI/ML pipeline encompasses four distinct processing streams: Disease Prediction Pipeline (ML.NET symptom classification), Differential Diagnosis Pipeline (multi-output probability distribution with clinical filtering), Risk Assessment Pipeline (gradient boosting continuous scoring with four-tier urgency), and Medical Image Analysis Pipeline (ONNX CNN with CLAHE preprocessing, NMS post-processing, and bounding box report generation). A feedback loop ensures prediction results feed back into the data pool for periodic model retraining.

Urgency Tier	Risk Score	Alert Response	Clinical Action
Emergency	Above 80	Same-day alert	Immediate clinical intervention required
Urgent	60 - 80	Within 24 hours	Priority consultation and further testing
Normal	30 - 60	Within 3 days	Routine follow-up scheduling
Follow-up	Below 30	Within 1 week	Standard monitoring and wellness check

Table 7: Four-Tier Risk Assessment Urgency Classification

8. AI Chatbot Architecture

The AI Chatbot is built on a pipeline architecture processing four input types through specialized pre-processing modules, enriching queries with RAG-retrieved patient context, generating responses through locally-hosted Ollama LLaMA 3, and applying safety guardrails before delivery. The entire chatbot system runs within Docker containers (or Kubernetes pods in production), ensuring isolation and complete data privacy.

Input Type	Processing Pipeline	Example Use Cases	Technology
Text Input	Tokenization, medical NER, symptom extraction, query classification	Ask about symptoms, drug dosages, clinical guidelines, differential diagnosis support	Ollama LLaMA 3 + NLP Pipeline
Image Input	Image upload, ONNX CNN inference, feature extraction, visual description generation	Upload X-ray/MRI/skin lesion for AI second opinion, compare with previous scans	ONNX Runtime + Ollama Vision
Voice Input	Audio capture, Whisper.cpp transcription, medical term correction, text routing	Dictate clinical observations, ask questions hands-free during procedures	Whisper.cpp + Ollama
Document Input	PDF/DOCX parsing, text extraction, structure recognition, content summarization	Upload lab reports, discharge summaries, referral letters for AI analysis	DocX + Ollama

Table 8: Chatbot Multi-Modal Input Processing

9. Database Design

The MAPS database is implemented in PostgreSQL 16 with 20 tables (three additional chatbot tables: ChatSessions, ChatbotMessages, ChatbotMemory; plus K8s health monitoring stored as structured JSON in HealthRecords). EF Core provides type-safe LINQ queries and automatic migrations. pgvector extension enables RAG vector similarity search. In the Kubernetes deployment, PostgreSQL runs as a StatefulSet with a 20Gi PersistentVolumeClaim, automated daily backup CronJob, and read replicas for scaling read-heavy query patterns.

Table	Purpose	Key Relationships
Users	All user accounts with role assignments	DoctorProfiles, PatientProfiles
DoctorProfiles	Doctor-specific data (specialization, license)	Users, Assignments, Prescriptions
PatientProfiles	Patient-specific data (blood group, DOB)	Users, Assignments, HealthRecords
Assignments	Doctor-Patient relationship mapping	Doctors, Patients, AuditLog
Appointments	Scheduling data with priority and status	Doctors, Patients, RiskAssessments
HealthRecords	Longitudinal clinical data entries	Patients, Doctors, ClinicalNotes
AIPredictions	Prediction results with confidence scores	Patients, Doctors, DifferentialDiagnoses
DifferentialDiagnoses	Multi-condition ranked diagnosis results	AIPredictions, SuggestedTests
RiskAssessments	Patient risk scores with trend tracking	Patients, Doctors, Appointments
ClinicalNotes	Free-text notes with NLP-extracted entities	HealthRecords, ExtractedEntities
ExtractedEntities	Structured medical entities from NLP	ClinicalNotes, HealthRecords
MedicalImages	Uploaded images with AI analysis results	Patients, Doctors, HealthRecords
Prescriptions	Medication orders with dosage and duration	Doctors, Patients, HealthRecords
ChatMessages	Real-time message persistence	Users (Sender/Receiver)
Feedback	Patient ratings and comments	Patients, Doctors
AuditLog	Tamper-proof action logging	Users (all actions)
Announcements	System-wide and targeted announcements	Admin (creator)
ChatSessions	AI chatbot session tracking	DoctorProfiles, ChatbotMessages
ChatbotMessages	Chatbot conversation messages with input type	ChatSessions, ChatbotMemory
ChatbotMemory	Vector embeddings for RAG context retrieval	DoctorProfiles, ChatSessions

Table 9: MAPS Database Schema — Table Definitions (20 Tables Including Chatbot)

10. Docker Containerization

NEW INFRASTRUCTURE — One-Command Deployment

MAPS is fully containerized using Docker with Docker Compose orchestration, providing consistent deployment across development, testing, and production environments.

The containerization strategy separates the platform into four network tiers — Frontend, Backend, AI, and Monitoring — each running in an isolated Docker network with controlled inter-container communication. This approach eliminates the 'it works on my machine' problem, simplifies deployment to a single command ('docker compose up -d'), and provides built-in service isolation and security through Docker network segmentation.

10.1 Container Inventory

Container	Image	Port(s)	Network	Volume
API	maps-api:latest (.NET 8)	5000 (internal)	Frontend + Backend	onnx_models (ro)
MVC Web	maps-web:latest (.NET 8)	8080 (internal)	Frontend	-
Nginx	nginx:alpine	443 (external)	Frontend	ssl_certs
PostgreSQL	postgres:16	5432 (internal)	Backend	pgdata
Redis	redis:7-alpine	6379 (internal)	Backend	redisdata
MinIO	minio/minio	9000 (internal)	Backend	miniodata
Ollama	ollama/ollama	11434 (internal)	AI	ollama_models
Whisper.cpp	maps-whisper:latest	8090 (internal)	AI	-
Grafana	grafana/grafana	3000 (internal)	Monitoring	grafana_data
Loki	grafana/loki	3100 (internal)	Monitoring	loki_data
Prometheus	prom/prometheus	9090 (internal)	Monitoring	-

Table 10: Docker Container Inventory

10.2 Docker Compose Configuration

The docker-compose.yml file defines all services, networks, and volumes required for the MAPS platform. The configuration specifies four custom Docker networks: maps-frontend (bridge mode, exposing Nginx on port 443), maps-backend (internal mode, no external access), maps-ai (internal mode, for Ollama and Whisper communication), and maps-monitoring (internal mode, for logging and metrics).

10.3 Deployment Workflow

The deployment workflow is streamlined to a three-step process:

- Clone the repository and configure the .env file with environment-specific settings (database passwords, JWT signing keys, Ollama model selection)
- Run 'docker compose build' to build the .NET API and MVC containers from the Dockerfile (multi-stage builds for minimal image size)
- Run 'docker compose up -d' to start all services in detached mode — the entire deployment completes within 5 minutes

Updates are performed using 'docker compose pull && docker compose up -d' which performs rolling updates with zero downtime for stateless services. Database migrations are executed automatically by the API container on startup using Entity Framework Core's automatic migration feature.

11. Kubernetes Orchestration

PRODUCTION INFRASTRUCTURE — Enterprise-Grade Scaling & Resilience

Kubernetes (K8s) is added as the production orchestration layer for MAPS, enabling horizontal pod autoscaling, zero-downtime rolling updates, self-healing deployments, and declarative infrastructure management via Helm charts. Docker Compose remains the recommended tool for local development and staging, while Kubernetes governs production workloads.

Kubernetes addresses the limitations of single-host Docker Compose deployments by distributing containerized workloads across a cluster of nodes, providing automatic failover, load balancing across multiple pod replicas, and cluster-wide secret management. For MAPS, this is particularly critical for the AI inference containers (Ollama, Whisper.cpp, ONNX) which are computationally intensive and benefit from GPU node scheduling and resource quotas.

11.1 Kubernetes Architecture Overview

K8s Component	MAPS Usage	Configuration Detail
Deployment	API, MVC Web, Nginx Ingress, Ollama, Whisper, Prometheus, Grafana	Declarative pod spec; restartPolicy: Always; multi-container pod for API + sidecar logging
StatefulSet	PostgreSQL, Redis, MinIO	Stable network identity; ordered pod startup; persistent volume per pod replica
HorizontalPodAutoscaler (HPA)	API Deployment, MVC Web Deployment	Scale 2→10 replicas on CPU >70% or memory >80%; 30-second cooldown period
Service (ClusterIP)	Internal inter-pod communication	API, PostgreSQL, Redis, Ollama, Whisper.cpp, Grafana, Loki all exposed as ClusterIP services
Service (LoadBalancer)	External traffic entry point	Nginx Ingress Controller exposed as LoadBalancer; single external IP/DNS for all traffic
Ingress (Nginx)	HTTP/HTTPS routing and TLS termination	Route /api/* → API service; / → MVC Web; /grafana/* → Grafana; cert-manager for TLS certs
ConfigMap	Non-secret environment configuration	Database connection strings (non-auth), Ollama model names, Hangfire config, CORS origins
Secret	Sensitive credentials management	JWT signing key, DB passwords, MinIO keys, Redis password — base64-encoded, RBAC-controlled
PersistentVolumeClaim (PVC)	Database and model storage	pgdata: 20Gi; miniodata: 50Gi; ollama_models: 30Gi; grafana_data: 5Gi — StorageClass: standard
Namespace	Environment isolation	maps-production, maps-staging, maps-monitoring — separate RBAC policies per namespace
ResourceQuota	Prevent resource exhaustion	CPU/memory limits per namespace; Ollama GPU resource limit; priority classes for AI workloads

K8s Component	MAPS Usage	Configuration Detail
NetworkPolicy	Pod-level network segmentation	API pods only reachable from Ingress; DB pods only reachable from API; AI pods isolated to AI namespace
CronJob	Scheduled background tasks	Risk score recalculation every 15 minutes; model performance monitoring daily; audit log archival weekly
ServiceAccount + RBAC	Kubernetes API access control	Dedicated ServiceAccount per deployment; Role bindings scoped to minimum required permissions

Table 11: Kubernetes Component Inventory for MAPS

11.2 Kubernetes Cluster Topology

The MAPS Kubernetes cluster is organized into three node pools optimized for different workload characteristics:

Node Pool	Workloads Scheduled	Recommended Spec	Notes
General (CPU)	API, MVC Web, Nginx, Redis, Prometheus, Grafana	4 vCPU / 8 GB RAM per node; min 2 nodes	Auto-scaled by Cluster Autoscaler; HPA manages pod count within this pool
Data (Stateful)	PostgreSQL, MinIO (StatefulSets)	8 vCPU / 16 GB RAM; NVMe SSD storage; min 1 node	Stateful workloads pinned to data pool via nodeSelector; PVCs attached per pod
AI / GPU	Ollama (LLaMA 3), Whisper.cpp, ONNX inference	8 vCPU / 32 GB RAM + GPU (optional); min 1 node	GPU-enabled nodes use nvidia.com/gpu resource requests; taint/toleration ensures only AI pods schedule here

Table 12: Kubernetes Node Pool Design

11.3 Helm Chart Structure

MAPS is packaged as a Helm chart enabling environment-specific deployments from a single templated codebase. The chart structure is as follows:

Helm Chart File	Contents / Purpose
charts/maps/Chart.yaml	Chart metadata: name, version, appVersion, dependencies (postgresql, redis Bitnami charts)
charts/maps/values.yaml	Default values: replica counts, image tags, resource limits, ingress host, secret references
charts/maps/values.prod.yaml	Production overrides: higher replica counts, resource limits, GPU node selectors, TLS config
charts/maps/values.staging.yaml	Staging overrides: single replicas, debug logging enabled, reduced resource limits
charts/maps/templates/api-deployment.yaml	API Deployment manifest with HPA, liveness/readiness probes, env from ConfigMap/Secret
charts/maps/templates/web-deployment.yaml	MVC Web Deployment manifest with replica count and Ingress routing rule

Helm Chart File	Contents / Purpose
charts/maps/templates/ollama-deployment.yaml	Ollama Deployment with GPU resource request, PVC for model weights, init container for model pull
charts/maps/templates/whisper-deployment.yaml	Whisper.cpp Deployment with audio processing resource limits
charts/maps/templates/postgres-statefulset.yaml	PostgreSQL StatefulSet with PVC template, backup CronJob, and pgvector extension init script
charts/maps/templates/minio-statefulset.yaml	MinIO StatefulSet with distributed mode config and PVC for medical image storage
charts/maps/templates/ingress.yaml	Nginx Ingress with TLS cert-manager annotation, path routing for API/Web/Grafana
charts/maps/templates/hpa.yaml	HorizontalPodAutoscaler for API and MVC Web targeting CPU and memory metrics
charts/maps/templates/networkpolicy.yaml	NetworkPolicy manifests enforcing pod-level isolation between frontend, backend, AI, monitoring namespaces
charts/maps/templates/rbac.yaml	ServiceAccount, Role, and RoleBinding definitions for each deployment
charts/maps/templates/configmap.yaml	Application configuration: Ollama endpoint, CORS, Hangfire settings, log level
charts/maps/templates/secrets.yaml	Kubernetes Secret template (references external secret manager or .env injection)

Table 13: MAPS Helm Chart File Structure

11.4 Deployment Workflow (Kubernetes)

The Kubernetes deployment workflow progresses through the following stages:

Step	Action	Detail
1	Build & Push Images	CI/CD pipeline (GitHub Actions) builds Docker images on push to main branch; tags with git SHA; pushes to container registry (Docker Hub / GitHub Container Registry)
2	Configure Values	Clone repository; copy values.prod.yaml; set ingress host, image tags, resource limits, and secret references for target environment
3	Apply Namespaces & RBAC	kubectl apply -f namespaces.yaml; creates maps-production, maps-staging, maps-monitoring namespaces with RBAC isolation
4	Install / Upgrade via Helm	helm upgrade --install maps-prod ./charts/maps -f values.prod.yaml -n maps-production — Helm diffs and applies only changed resources
5	Verify Rollout	kubectl rollout status deployment/maps-api -n maps-production; kubectl get pods --all-namespaces to verify all pods Running
6	Database Migration	API pod init container runs 'dotnet ef database update' automatically on first startup; subsequent pods skip if migrations current
7	Smoke Test	Automated health check hits /api/health and /api/system/containers endpoints; verifies all containers healthy
8	HPA Activation	Prometheus scrapes metrics; HPA begins monitoring CPU/memory; scales pods as load increases

Table 14: Kubernetes Deployment Workflow

11.5 Horizontal Pod Autoscaling (HPA) Policy

Deployment	Min Replicas	Max Replicas	Scale-Up Trigger	Scale-Down Policy
maps-api	2	10	CPU >70% OR Memory >80%	Scale down after 5 min below threshold; max 1 pod/min
maps-web (MVC)	1	5	CPU >60%	Scale down after 3 min below threshold
ollama (LLM)	1	3	GPU utilization >80% (if GPU node)	Manual scale-down only (model load time >2 min)
whisper-cpp	1	4	CPU >75%	Scale down after 10 min; audio jobs queue-based
maps-monitoring	1	1	N/A (single replica)	No autoscaling; monitoring must always be up

Table 15: HPA Policy Per Deployment

11.6 Kubernetes vs Docker Compose — When to Use Which

Aspect	Docker Compose	Kubernetes (K8s)
Environment	Local development, staging, demos	Production, load testing, CI validation
Deployment command	docker compose up -d	helm upgrade --install maps ./charts/maps
Scaling	Manual (docker compose scale api=3)	Automatic via HPA; Cluster Autoscaler for nodes
Self-healing	Container restart on failure	Pod rescheduling across healthy nodes
Load balancing	Nginx round-robin	K8s Service + Ingress; L4/L7 balancing
Secret management	.env file (local only)	Kubernetes Secrets encrypted in etcd + RBAC
Rolling updates	Recreate strategy (brief downtime)	RollingUpdate: maxSurge=1, maxUnavailable=0
Resource limits	Optional (docker-compose.yml)	Enforced via ResourceQuota + LimitRange
Multi-node support	No (single host)	Yes (distributed across cluster nodes)
Complexity	Low (ideal for development)	Higher (production-grade; requires K8s knowledge)

Table 16: Docker Compose vs Kubernetes — Selection Guide

11.7 Kubernetes Networking & Security

Security in the Kubernetes layer is enforced through multiple complementary mechanisms:

- NetworkPolicy — Pod-level firewall rules ensure only authorized pods communicate. API pods accept traffic only from Ingress; PostgreSQL accepts only from API pods; Ollama accepts only from API pods within the maps-ai namespace.

- TLS Everywhere — cert-manager issues Let's Encrypt TLS certificates for the Nginx Ingress; internal pod communication uses mTLS enforced by service mesh (optional Linkerd integration).
- Secrets Management — All sensitive credentials (JWT keys, DB passwords, MinIO keys) are stored as Kubernetes Secrets, RBAC-restricted, and optionally integrated with HashiCorp Vault via External Secrets Operator.
- RBAC — ServiceAccounts with least-privilege Role bindings for each deployment; no deployment uses the default ServiceAccount.
- Pod Security Standards — Pods run as non-root users; readOnlyRootFilesystem enforced; privilege escalation disabled; seccompProfile: RuntimeDefault applied.

12. Security Architecture

The MAPS platform implements a multi-layered security architecture spanning authentication, authorization, data encryption, audit logging, Docker network isolation, and Kubernetes pod-level security. Security measures create a defense-in-depth strategy addressing threats at every level of the technology stack.

Security Layer	Mechanism	Detail
Authentication	ASP.NET Core Identity + JWT Bearer	15-min access tokens, 7-day refresh rotation; bcrypt password hashing; Registration Lock
Authorization	Role-based Claims + Policy-based API Guards	JWT embeds role claims; [Authorize(Roles)] enforced on every controller action
Transport Encryption	TLS 1.3 via Nginx (Docker) / K8s Ingress	All HTTP, WebSocket, file upload traffic encrypted; cert-manager automates TLS renewal in K8s
Data at Rest	PostgreSQL TDE + AES-256 MinIO + Column Encryption	PII fields encrypted at column level via EF Core value converters; K8s Secrets encrypted in etcd
Network Isolation	Docker Networks (dev) + K8s NetworkPolicy (prod)	Backend, AI, Monitoring networks have no external access; K8s NetworkPolicy enforces pod-level rules
Audit Trail	Append-only AuditLog table + Serilog + Grafana Loki	Every action logged; tamper-proof; Grafana dashboards with automated anomaly alerting
K8s Pod Security	Pod Security Standards + RBAC + ServiceAccounts	Non-root users; readOnlyRootFilesystem; no privilege escalation; seccompProfile: RuntimeDefault

Table 17: MAPS Security Architecture Summary

13. API Design

The MAPS platform exposes a RESTful API layer built on ASP.NET Core Web API serving as the communication backbone. All endpoints follow consistent naming conventions, standard HTTP methods for CRUD operations, and standardized response formats. Swagger/OpenAPI auto-generates interactive API documentation.

Endpoint Group	Key Routes	Access Level
Authentication	POST /api/auth/login, /refresh, /register	Public + Admin Approval
User Management	GET/POST/PUT/DELETE /api/users	Admin Only
Doctor-Patient Assignment	GET/POST/PUT/DELETE /api/assignments	Admin Only
AI Predictions	POST /api/predictions, GET /api/predictions/{id}	Doctor / Admin
Differential Dx	POST /api/predictions/differential	Doctor / Admin
Risk Assessment	GET /api/risks/patient/{id}, GET /api/risks/alerts	Doctor / Admin

Endpoint Group	Key Routes	Access Level
Clinical Notes	POST/GET /api/notes, GET /api/notes/{id}/entities	Doctor / Admin
Medical Images	POST /api/images/analyze, GET /api/images/{id}	Doctor / Admin
Chatbot (NEW)	POST /api/chatbot/query, /chatbot/session, GET /api/chatbot/history	Doctor Only
Voice Dictation (NEW)	POST /api/voice/transcribe	Doctor Only
Drug Interactions (NEW)	POST /api/drugs/check-interactions	Doctor Only
Literature Search (NEW)	POST /api/literature/search	Doctor Only
Appointments	GET/POST/PUT/DELETE /api/appointments	Doctor / Patient / Admin
Prescriptions	GET/POST/PUT /api/prescriptions	Doctor / Admin
Chat	SignalR Hub /api/chat, GET /api/chat/history/{userId}	Doctor / Patient / Admin
Feedback	POST /api/feedback, GET /api/feedback	Patient / Admin
Audit Log	GET /api/audit, GET /api/audit/{userId}	Admin Only
Analytics	GET /api/analytics/dashboard, /predictions, /users	Admin Only
Docker Status (NEW)	GET /api/system/containers, /health	Admin Only
K8s Cluster Status (NEW)	GET /api/system/pods, /api/system/deployments, /api/system/hpa	Admin Only

Table 18: MAPS API Endpoint Catalog (Including Chatbot, Docker & Kubernetes Endpoints)

14. Scope and Expected Outcomes

14.1 In Scope

- Three-role system: Admin, Doctor, Patient with hierarchical access control
- Multi-disease prediction engine covering Diabetes, Heart Disease, Pneumonia, Brain Tumour, and Skin Cancer
- Three prediction modalities: structured dataset input, medical image upload, and text prompt via Ollama LLM
- AI Personalized Chatbot for doctors with four input modalities: text, image, voice, and document
- RAG-enhanced chatbot context using pgvector for vector similarity search against patient health records
- Voice-to-Text clinical dictation using Whisper.cpp with medical term correction
- Drug Interaction AI Checker with severity classification and patient-specific risk assessment
- Medical Literature Search using RAG architecture for clinical guidelines and drug monographs
- Docker containerization with Docker Compose orchestration for one-command development/staging deployment
- **Kubernetes orchestration with Helm charts, HPA, NetworkPolicy, and rolling updates for production deployment**
- Hybrid deployment: WinForms desktop client and ASP.NET Core MVC web client sharing a single backend API
- Real-time SignalR chat with file sharing supporting images, PDF, and DOCX attachments
- Mandatory post-consultation feedback with Admin analytics dashboard and NLP sentiment analysis

- First-run registration lock for Admin and Doctor roles with double-layer UI and API enforcement
- JWT authentication and role-based API authorisation with short-lived access tokens and refresh token rotation
- PDF report generation per patient and per prediction using QuestPDF (MIT-licensed, free)
- PostgreSQL database with Entity Framework Core ORM, pgvector extension, and full audit trails
- NLP-driven clinical note summarization and medical entity extraction with F1 score above 0.80
- ONNX-based medical image classification pipeline with CNN model inference in C# runtime
- Continuous risk assessment with four-tier urgency classification and proactive alerting
- All free and open-source technology stack with zero licensing costs
- Grafana Loki + Serilog for structured logging; Prometheus for Kubernetes metrics monitoring
- MinIO for self-hosted S3-compatible medical image and document storage

14.2 Out of Scope (Initial Release)

- Native mobile application (iOS / Android)
- Integration with hospital PACS or HL7/FHIR electronic health record systems
- Online payment or insurance billing modules
- Training of deep learning models (models pre-trained in Python; ONNX inference only in C#)
- Service mesh (Linkerd/Istio) — optional future enhancement for mTLS between pods
- Multi-region Kubernetes cluster federation

14.3 Expected Outcomes

A fully functional hybrid C# medical AI prediction system deployable via Docker Compose (single command, development/staging) and Kubernetes via Helm (production-grade, auto-scaling). This includes:

- Multi-disease prediction capability covering Diabetes, Heart Disease, Pneumonia, Brain Tumour, and Skin Cancer through three input modalities
- A personalized AI chatbot for doctors supporting text, image, voice, and document inputs with RAG-enhanced clinical context
- Voice-to-text clinical dictation for hands-free note creation
- Intelligent drug interaction checking with patient-specific severity assessment
- Medical literature search grounded in clinical guidelines through RAG architecture
- Secure real-time communication channel between doctors and patients with file sharing
- Complete Docker containerization with one-command deployment for development environments
- **Production-ready Kubernetes orchestration with auto-scaling, rolling updates, self-healing, and encrypted secrets management**
- Professional PDF report generation using QuestPDF
- A scalable, maintainable codebase ready for future expansion including mobile client, FHIR integration, and Kubernetes service mesh

15. Conclusion

The MAPS platform represents a comprehensive approach to AI-augmented healthcare that prioritizes clinical safety, professional oversight, and patient-centered care. By implementing a strict role hierarchy where Administrators govern the platform (including Docker and Kubernetes infrastructure), Doctors leverage advanced AI tools including a personalized multi-modal chatbot for clinical decision support, and Patients access health information through a mediated secure portal, MAPS ensures that AI serves as an assistive technology under human clinical control.

The infrastructure architecture is designed for the full software development lifecycle: Docker Compose for rapid local development and demos, and Kubernetes with Helm for production deployment with enterprise-grade resilience. The Kubernetes layer adds horizontal pod autoscaling for the API and AI inference containers, zero-downtime rolling updates, pod-level NetworkPolicy security, and declarative secret management — capabilities that are essential for a healthcare platform handling sensitive patient data at scale.

The migration to an entirely free and open-source technology stack (PostgreSQL, QuestPDF, MinIO, Ollama, Grafana Loki, Prometheus, Kubernetes, Helm) demonstrates that enterprise-grade healthcare technology can be built without licensing costs or vendor dependency. Docker containerization with Docker Compose and Kubernetes orchestration ensures consistent deployment, network isolation, and service manageability across every environment from a developer's laptop to a production cluster.

The platform's modular architecture supports future expansion including wearable device integration for continuous vital sign monitoring, telemedicine video consultations, predictive population health analytics, Kubernetes service mesh (Linkerd) for mTLS between pods, multi-region cluster federation for disaster recovery, and integration with national electronic health record systems. MAPS demonstrates that AI in healthcare can be both powerful and responsible, combining cutting-edge machine learning with the clinical governance framework and modern DevOps infrastructure necessary to ensure patient safety, professional accountability, and operational excellence.

Medical AI Prediction System (MAPS) — Final Project Proposal 2026

BSCS-6-B · KICSIT Computer Science Department · Sir Uzair Janjua

All technologies are free and open-source · Docker · Kubernetes · ML.NET · Ollama